



OASIS Security Report

SQL Injection Security Analysis

Date: 2026-06-12 01:26:17 — **Model:** llama3.2:3b

Vulnerability: SQL Injection

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **7** file(s) for SQL Injection.

[Return to table of contents](#)

Statistics

Files analyzed: 7 **Total findings:** 8



Medium

0

Low

0

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol	0.635
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.612
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.605
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.603
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.602
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.601
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.591

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol

Similarity score: 0.635

Source lines: 1-70

► Finding 1.1: SQL Injection (High)

Vulnerable code

```
function borrow(uint256 borrowAmount) external {  
    // ...  
    require(_token.transfer(msg.sender, borrowAmount), "Transfer failed");  
}
```

Explanation

User input (`borrowAmount`) is directly used in the SQL query without parameterization.

Impact

An attacker could inject malicious SQL code to steal or manipulate data.

Entry point

/borrow(uint256)

HTTP methods

POST

Parameters

borrowAmount

Exploitation steps

- An attacker can construct a malicious `borrowAmount` value to execute arbitrary SQL queries.

Example payloads

- "123" OR 1=1 --

HTTP raw requests (Burp / ZAP)

```
POST /borrow(uint256) HTTP/1.1  
Host: <target>
```

Content-Type: application/json

```
{"amount": "123 OR 1=1--"}
```

File 2: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.612

Source lines: 1-61

► Finding 2.1: SQL Injection (Critical)

Vulnerable code

```
uint8 tot; address owner;
```

Explanation

User input is directly used in query without parameterization.

Impact

Data theft, data loss, or system compromise possible

Entry point

/play(uint)

Execution path

```
play -> players[tot] = Player(msg.sender, number); andTheWinnerIs() (with  
n%2==0/1)
```

HTTP methods

POST

Parameters

number, msg.sender

Exploitation steps

- Submit a crafted value for 'number' to force a certain player win condition
- Use a large number to overflow the uint8 tot variable

Example payloads

- 10000
- \x41\x42\x43

Conditions

Owner is authenticated and msg.sender can submit any value for 'number'.

Remediation

Parameterize queries using :number or :addr, and validate user input.

Secure example

```
function play(uint number) payable {if (msg.value != 1 ether) throw;  
players[tot] = Player(msg.sender, number); tot++; ...}
```

File 3: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.605

Source lines: 1-286

► Finding 3.1: SQL Injection Vulnerability (Critical)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner  
returns (bytes32 o_hash)
```

Explanation

The use of raw input directly in the query without parameterization makes it vulnerable to SQL injection.

Impact

An attacker can inject malicious SQL code to steal or modify sensitive data.

Entry point

/execute

HTTP methods

POST

Parameters

_to, _value, _data

Exploitation steps

- Send a crafted request with a malicious operation hash.
- Exploit the vulnerability to gain unauthorized access.

Example payloads

- ' OR 1=1 --

HTTP raw requests (Burp / ZAP)

```
POST /execute HTTP/1.1  
Host: example.com  
Content-Length: 1000
```

```
' OR 1=1 --
```

Source lines: 96-414

► Finding 3.1: Vulnerable Code (Critical)

Vulnerable code

```
(uint ownerIndex) = uint(msg.sender); clearPending(); m_owners[ownerIndex] =  
uint(_to);
```

Explanation

The use of `uint(ownerIndex)` without proper sanitization allows an attacker to manipulate the index of `\_owners` array.

Impact

Arbitrary code execution and data tampering possible.

Entry point

```
/initMultiowned(address[] _owners, uint _required)
```

Exploitation steps

- Find a valid owner index.
- Manipulate the index to point to a desired value in `\_owners` array.

File 4: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.603

Source lines: 1-85

► Finding 4.1: Vulnerability found (Critical)

Vulnerable code

```
function ETH_VAULT(address _log) public { TransferLog = Log(_log); }
```

Explanation

This function uses string concatenation in SQL query without parameterization, allowing an attacker to inject malicious SQL code.

Impact

Unauthorized access to user accounts and potentially sensitive data

Entry point

/transfer

HTTP methods

POST

Parameters

_log

Exploitation steps

- Pass a malicious address as the _log parameter

Example payloads

- ' OR 1=1 --

HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1
Host: <target_host>
Log: <malicious_address> OR 1=1 --
```

```
Host: <target_host>
```

0

File 5: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.602

Source lines: 1-31

► Finding 5.1: Vulnerability found (High)

Vulnerable code

```
receiver.transfer(amount);
```

Explanation

Direct use of tx.origin without parameterization makes it vulnerable to SQL Injection.

Impact

Data tampering and potential authentication bypass

Entry point

sendTo(address receiver, uint amount) public

Execution path

```
tx.origin -> receiver.transfer(amount);
```

Parameters

receiver, amount

Exploitation steps

- Find vulnerable contract
- Send malicious tx to exploit

Example payloads

- malicious uint amount

Conditions

tx.origin == owner

Remediation

Use parameterized queries or prepared statements.

Secure example

```
require(tx.origin == owner) { receiver.transfer(amount); }
```

File 6: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.601

Source lines: 1-35

► Finding 6.1: Vulnerability found (Critical)

Vulnerable code

```
bool res = msg.sender.call.value(amount)();
```

Explanation

Directly using user input (msg.sender) without parameterization in a function call.

Impact

Authenticating attacker could drain funds from the DAO.

Entry point

/withdraw

Execution path

```
call -> update credit amount -> return credit value
```

HTTP methods

POST

Parameters

msg.sender, amount

Exploitation steps

- Authenticate with DAO via fraudulent request

Example payloads

- malicious address

Conditions

User is authenticated and has sufficient balance.

Remediation

Use parameterized queries or prepared statements for all user input functions

Secure example

```
bool res = msg.sender.call.value(amount, uint256(amount)); credit[msg.sender]  
-= amount;
```

File 7: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.591

Source lines: 1-39

► Finding 7.1: SQL Injection (High)

Vulnerable code

```
(require(balances[msg.sender] >= _weiToWithdraw) &&  
msg.sender.call.value(_weiToWithdraw()))
```

Explanation

The use of `call` without parameterized queries allows an attacker to inject malicious SQL.

Impact

Data tampering or theft by manipulating the `call` function with malicious input.

Entry point

/withdrawFunds

Execution path

```
msg.sender.call.value(_weiToWithdraw)  
msg.sender  
now >= lastWithdrawTime[msg.sender] + 1 weeks  
balances[msg.sender] -= _weiToWithdraw;
```

Parameters

_weiToWithdraw

Exploitation steps

- Send a crafted input for _weiToWithdraw that exploits the vulnerability.

Example payloads

- 100000 ether

Conditions

The attacker must be able to reach the /withdrawFunds endpoint while authenticated.

Remediation

Use parameterized queries or prepared statements for the ``call`` function.

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)